데이터구조 3장 레포트

일체의 부정 행위 없이 스스로의 노력으로 작성한 레포트임을 확인합니다.

1.(1) 코드 및 주석

```
#include<stdio.h>
int main() {
        int day[12] = { 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 }; //배열 만들기

        for (int i =0; i <12; i++) // for문을 사용하여 1월부터 12월까지 출력
        {
                printf("%d월은 %d일까지 있습니다. \n", (i +1), day[i]);
        }
        return 0; // 0을 반환하며 종료
}
```

1.(1) 출력화면 및 설명

```
1월은 31까지 있습니다.
2월은 29까지 있습니다.
3월은 31까지 있습니다.
4월은 30까지 있습니다.
5월은 31까지 있습니다.
6월은 30까지 있습니다.
7월은 31까지 있습니다.
8월은 31까지 있습니다.
9월은 30까지 있습니다.
10월은 31까지 있습니다.
11월은 30까지 있습니다.
12월은 31까지 있습니다.

C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 3112개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용
하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

for문을 활용해서 day[12] 배열을 순서대로 출력하였다.

1.(2) 코드 및 주석

```
#include<stdio.h>

void matrix_diff(int* a, int* b, int* c) //(a,b,c)를 받는 포인터 매개변수 (*a,*b,*c)
{
        for (int i = 0; i < 10; i++) //for문을 이용해서 배열의 차를 구하기
       {
                c[i] = a[i] - b[i]; //a와 b의 차를 c에 저장
                printf("c[%d] = %d \n", i, c[i]); //배열 c 출력
        }
}
int main()
{
        int a[10] = { 10,20,30,40,50,60,70,80,90,100}; //배열 a 생성
        int b[10] = { 100,200,300,400,500,600,700,800,900,1000 }; //배열 b 생성
        int c[10] = { 0 }; //배열 c 생성

        matrix_diff(a, b, c); //함수 matrix_diff 실행
        return 0;
}
```

1.(2) 출력화면 및 설명

```
c[0] = -90
c[1] = -180
c[2] = -270
c[3] = -360
c[4] = -450
c[5] = -540
c[6] = -630
c[7] = -720
c[8] = -810
c[9] = -900

C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 8개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

크기가 10인 배열 a와 b를 생성하였다. 그 후 포인터 매개변수를 이용하여 matrix_diff로 이동하여 for문을 이용하여 배열 a와 b의 차를 구한 값을 배열 c에 저장한다. 그 후 배열 c 값을 출력한다.

1.(3) 코드 및 주석

```
#include<stdio.h>
#include <stdlib.h> //rand함수를 활용하기 위해서 선언하였다.

int main()
{
        int random[10]; //크기 10인 배열을 생성하였다.
        int max, min; //최댓값과 최솟값을 넣을 변수를 생성하였다.
        srand(time(NULL));  //난수값이 시간에 따라 변화하도록 하였다.

        for (int i = 0; i < 10; i++)
        {
                random[i] = rand(); //난수값을 배열에 넣는 과정.
                if (i == 0)
                  // i가 0일때의 배열의 값을 max와 min에 넣어 비교할 상황을 만든다.
                {
                        max = random[i];
                        min = random[i];
                }
                if (i > 0)
                {
                        if (random[i] > max)
                        {
                                max = random[i];
                           //random[i]가 max보다 클 경우 max에 random[i]값이 들어간다.
                        }
                        else if (random[i] < min)
                        {
                                min = random[i];
                           //random[i]가 min보다 작을 경우 min에 random[i]값이 들어간다.
                        }

                }
        }
        printf("최댓값은 %d이고, 최솟값은 %d이다.", max, min); // 출력

        return 0;
}
```

1.(3) 출력화면 및 설명

```
최댓값은 31133이고, 최솟값은 924이다.
C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 16916개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

크기가 10인 배열, 최댓값,최솟값을 만든 후, srand(time(NULL));을 이용하여 시간의 변화에 따라 난수의 값이 변하게 설정을 하였다. for문을 이용하여 배열에 난수값을 넣고, if문을 이용하여 random[i] 값이 max값보다 큰 경우 max값에 넣는 방식과 random[i]값이 min값보다 작은 경우 min값에 넣는 방식을 이용하여 최댓값과 최솟값을 구해서 결과를 출력하였다.

1.(4) 코드 및 주석

```
#include <stdio.h>
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable:4996)

void search(int *x, int y, int *z) //메인 함수에 값을 가져가기 위해서 x와 z는 포인터 사용.
{
    for (int i = 0; i < 10; i++) //for문을 이용해서 배열 안에 값이 있는지 찾기
    {
        if (x[i] == y) //배열과 값이 일치하는경우
        {
            *z = i; // *z(number주소값)에 i를 넣는다.
        }
    }
}
int main()
{
    int arr[10] = { 23, 45, 12, 34, 65, 25, 89, 61, 26, 11 }; //크기가 10인 1차원 배열
    int value = 0; //찾고자 하는 값
    int number = -1; //요소의 위치(배열은 0부터 시작해서 기본값을 -1로 설정함.
    scanf("%d", &value); //찾고자 하는 값을 받는다.
    search(arr, value, &number); //임의의 값의 위치를 찾는 함수 search()

    if (number == -1) //number값이 안바뀔 경우
    {
        printf("%d는 배열 안에 없습니다.", value);
    }
    else //number 값이 바뀔 경우
    {
        printf("%d는 배열의 %d번째 있습니다.", value, number+1);
    }

    return 0;
}
```

1.(4) 출력화면 및 설명

```
12
12는 배열의 3번째 있습니다.
C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 12408개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

크기가 10인 임의의 값이 있는 arr[10]을 만들고, 찾고자 하는 값을 넣을 value변수와 요소의 위치 값을 넣을 number를 만든다.(number를 -1로 한 이유는 배열은 0부터 시작하기 때문이다.) 그 이후 scanf로 value값을 받은 후 search함수로 가서 for문과 if문을 활용하여 배열 안에 찾고자 하는 값과 일치하는 값을 찾아서 포인터를 이용해서 주소 안에 넣어준 후, main함수로 돌아와서 결과값(찾고자 하는 값의 위치)을 출력한다.

1.(5) 코드 및 주석

```
#include <stdio.h>
#define LIST_SIZE 10 //LIST_SIXZE값을 10으로 고정하였다.
int list[LIST_SIZE] = { 23, 45, 12, 34, 65, 25, 89, 61, 26, 11 }; //1차원 배열 생성
main()
{
int i;
for(i=0;i<LIST_SIZE;i++) //배열의 크기만큼 for문으로 반복
{
printf("list[%d]의 주소 = %p\n",i,&list[i]); //%p는 변수의 주소를 출력하는 포맷이라서  배열의 주소가 출력된다. 또한 list[i]앞에 &를 붙여서 주소를 출력하게 하였다.
}
}
```

1.(5) 출력화면 및 설명

```
list[0]의 주소 = 00007FF74E7DC000
list[1]의 주소 = 00007FF74E7DC004
list[2]의 주소 = 00007FF74E7DC008
list[3]의 주소 = 00007FF74E7DC00C
list[4]의 주소 = 00007FF74E7DC010
list[5]의 주소 = 00007FF74E7DC014
list[6]의 주소 = 00007FF74E7DC018
list[7]의 주소 = 00007FF74E7DC01C
list[8]의 주소 = 00007FF74E7DC020
list[9]의 주소 = 00007FF74E7DC024

C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 22392개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

%d를 사용하였으면 배열 안에 들어가 있는 정수들이 출력되었겠지만, %p와 list[i] 앞에 &를 사용하여 배열의 주소가 출력되었다.

1.(6) 코드 및 주석

```
#include < stdio.h>
void printArrayAddress(int [][3]); //printArrayAddress함수 선언
//
void main(void)
{
int array[2][3] = {{1, 2, 3}, {4, 5, 6}}; //2차원 배열 생성
printf("배열 array의 출력 : \n");
printArrayAddress(array); //printArrayAddress함수에 배열이름을 넘겨줌
}
//
void printArrayAddress(int a[][3])
{
int i, j;
for(i=0; i<=1; i++) //이중 for문을 이용해서 2차원 배열을 출력함.
{
for(j=0; j<=2; j++)
printf("%p ", &a[i][j]);
printf("\n");
}
}
```

1.(6) 출력화면 및 설명

```
배열 array의 출력 :
000000C97D9AF818 000000C97D9AF81C 000000C97D9AF820
000000C97D9AF824 000000C97D9AF828 000000C97D9AF82C

C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 8268개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

array배열을 만든 후 printArrayAddress함수로 이동하여 2중 for문을 이용해서 배열을 출력한다. 출력할 때는 %p와 &a[i][j]라서 배열의 주소가 출력이 된다.

2.(1)~(3) 코드 및 주석

```
#include <stdio.h>
#include <math.h>
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable:4996)

typedef struct student {
    char name[100]; //크기가 100인 학생의 이름
    int student_number; //학번
    int score; //성적
} student;

int get_mean(student add[], int number) //student 구조체와 학생 수를 매개변수로 받는다.
```

```
{
    int back = 0; //총합을 가질 back이라는 변수를 만든다.
    for (int i = 0; i < number; i++)
    {
        back = back + add[i].score; //for문을 이용하여 학생들의 성적을 back에 축적한다.
    }
     return back / number; //학생들의 총합을 학생수로 나눈 평균값을 반환한다.
}

float get_deviation(student add[], int number) //student 구조체와 학생 수를 매개변수로 받는다.
{
    float x = 0; //총합을 가질 x라는 변수를 만든다.
    float y = 0; //
    float average = 0; //평균을 가질 average 변수를만든다.
    for (int i = 0; i < number; i++)
    {
        x = x + add[i].score;
    }
    average= x / number; //get_mean함수와 같은방식으로 평균을 구해서 average에 넣는다.
    x = 0; //총합이 필요없기에 0값을 넣고 y값을 축적하여 더하는데 사용한다.
    for (int j = 0; j < number; j++)
    {
        y = pow(add[j].score -average,2); //(평균-성적)^2을 y에 저장한다.
        x = x + y; //y값을 x에 축적한다.
    }
    return sqrt(x / number); //sqrt함수를 이용하여 표준편차를 구해서 반환한다.
}

int main()
{
    student  students[1000] = { 0 }; //구조체 student의 배열 students[1000]을 만든다.
    int number = 0; //학생들의 숫자.
    scanf("%d", &number);
    for (int i = 0; i < number; i++) //학생들의 숫자만큼 입력받는다.
    {
        scanf("%s %d %d", students[i].name, &students[i].student_number, &students[i].score); // scanf로 입력받아 student구조체에 각각 저장한다.
    }
    printf("평균은 :%d\n", get_mean(students, number)); //평균을 구하는 get_mean으로 이동하여 값을 구하여 출력한다.
    printf("표준편차는 :%f", get_deviation(students, number)); //표준편차를 구하는 get_deviation으로 이동하여 값을 구하여 출력한다.
    return 0;
}
```

2.(1)~(3) 출력화면 및 설명

```
3
cha 20201518 65
han 20201519 80
gyeol 20201520 95
평균은 :80
표준편차는 :12.247449
C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 27060개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

구조체 student를 선언하여 학생의 이름과 학번 성적을 저장할 배열과 변수를 만든다. main함수에서 구조체 student의 배열 students[1000]를 만들어서 학생의 정보를 저장할 공간 1000개를 만든다. number변수에 학생들의 숫자를 입력받은 후 for문으로 학생 숫자만큼 학생의 이름, 학번, 점수를 입력받아 구조체에 저장한다. 그 후 get_mean함수와 get_deviation함수를 이용하여 각각 평균과 표준편차를 구한 후 반환하여 출력한다.

2.(4) 코드 및 주석

```
#include <stdio.h>
#include <math.h> //루트 기능의 함수를 사용하기 위해서 선언.
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable:4996)

typedef struct student {
    char name[100]; //크기가 100인 학생의 이름
    int student_number; //학번
    int score; //성적
} student;

int get_mean(student *add, int number) //student 구조체와 학생 수를 매개변수로 받는다.
{
    int back = 0; //총합을 가질 back이라는 변수를 만든다.
    for (int i = 0; i < number; i++)
    {
        back = back + (add + i)->score; //for문을 이용하여 학생들의 성적을 back에 축적한다.
    }
     return back / number; //학생들의 총합을 학생수로 나눈 평균값을 반환한다.
}

float get_deviation(student *add, int number) //student 구조체와 학생 수를 매개변수로 받는다.
{
    float x = 0; //총합을 가질 x라는 변수를 만든다.
    float y = 0; //
    float average = 0; //평균을 가질 average 변수를만든다.
    for (int i = 0; i < number; i++)
    {
        x = x + (add + i)->score;
    }
    average= x / number; //get_mean함수와 같은방식으로 평균을 구해서 average에 넣는다.
```

```
    x = 0; //총합이 필요없기에 0값을 넣고 y값을 축적하여 더하는데 사용한다.
    for (int j = 0; j < number; j++)
    {
        y = pow((add + j)->score-average,2); //(평균-성적)^2을 y에 저장한다.
        x = x + y; //y값을 x에 축적한다.
    }
    return sqrt(x / number); //sqrt함수를 이용하여 표준편차를 구해서 반환한다.
}

int main()
{
    student  students[1000] = { 0 }; //구조체 student의 배열 students[1000]을 만든다.
    int number = 0; //학생들의 숫자.
    scanf("%d", &number);
    for (int i = 0; i < number; i++) //학생들의 숫자만큼 입력받는다.
    {
        scanf("%s %d %d", students[i].name, &students[i].student_number, &students[i].score); // scanf로 입력받아 student구조체에 각각 저장한다.
    }
    printf("평균은 :%d\n", get_mean(students, number)); //평균을 구하는 get_mean으로 이동하여 값을 구하여 출력한다.
    printf("표준편차는 :%f", get_deviation(students, number)); //표준편차를 구하는 get_deviation으로 이동하여 값을 구하여 출력한다.
    return 0;
}
```

2.(4) 출력화면 및 설명

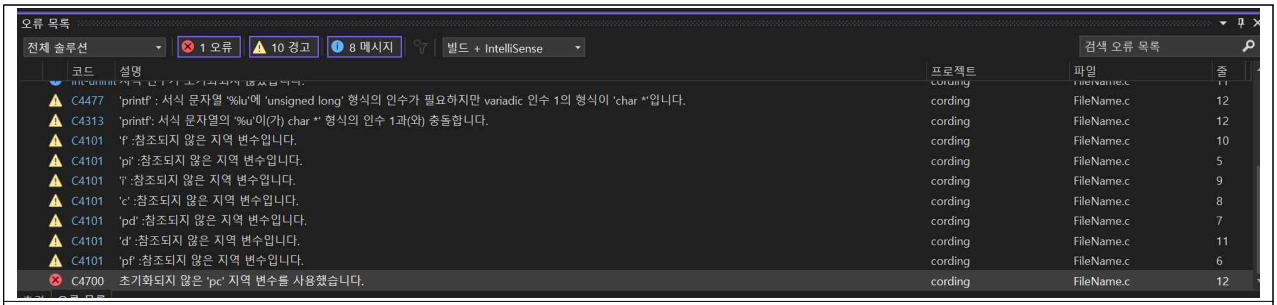
```
3
a 1 80
b 2 60
c 3 78
평균은 :72
표준편차는 :8.993825
C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 10880개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

2(1)~(3)까지 식에서 매개변수만 배열에서 포인터로 바꾼 것이기 때문에 결과값의 차이는 없다.

3.(1) 코드 및 주석

```
#include <stdio.h>
void main()
{
char *pc; //문자형 포인터 *pc를 만든다.
int *pi; //int형 포인터 *pi를 만든다.
float *pf; // float형 포인터 *pf를 만든다.
double *pd; //double형 포인터 *pd를 만든다.
char c; //문자형 변수 c를 만든다.
int i; //int 형 변수 i를 만든다.
float f; //float형 변수 f를 만든다.
double d; //double형 변수 d를 만든다.
printf("pc = %lu\n", pc); //long unsigned 변수 형태를 출력하는 식이다.
}
```

3.(1) 출력화면 및 설명

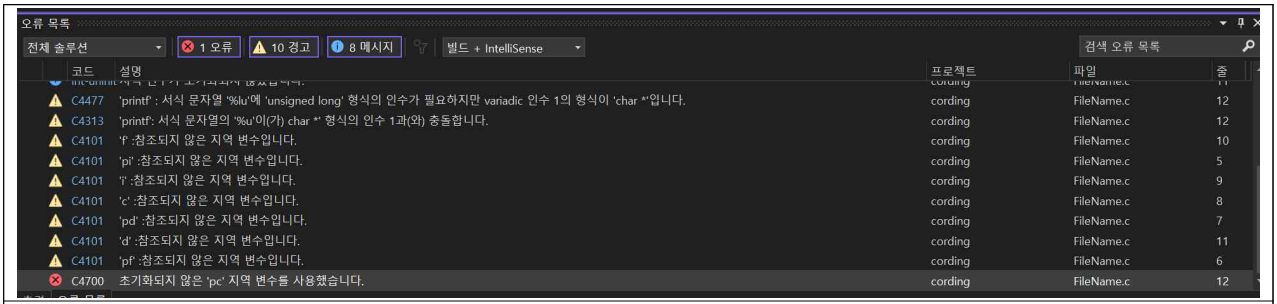


printf에서 %lu는 long unsigned형 변수를 받는데 pc는 문자형이기 때문에 오류가 발생한다.
또한 선언된 모든 변수와 포인터들이 초기화가 되어있지 않아 쓰레기값이 있다.

3.(2) 코드 및 주석

```
#include <stdio.h>
void main()
{
char *pc; //문자형 포인터 *pc를 만든다.
int *pi; //int형 포인터 *pi를 만든다.
float *pf; // float형 포인터 *pf를 만든다.
double *pd; //double형 포인터 *pd를 만든다.
char c; //문자열 변수 c를 만든다.
int i; //int 형 변수 i를 만든다.
float f; //float형 변수 f를 만든다.
double d; //double형 변수 d를 만든다.
printf("pc = %lu\n", pc); //long unsigned 변수 형태를 출력하는 식이다.
*pc = 100; //*pc가 바라보는 곳에 100값을 넣는다. 하지만 바라보는 주소가 없기 때문에 잘못된 식
}
```

3.(2) 출력화면 및 설명



결과값에는 변화가 없이 오류가 난다. 출력 후에 *pc =100;을 하든 출력 전에 *pc =100;를 하든 결과값은 달라지지 않을 것이다. *pc = 100;이라는 식은 *pc를 만들때부터 주소가 없었기 때문에 잘못된 식이다.

3.(3) 코드 및 주석

```
#include <stdio.h>
void main()
{
char *pc; //문자형 포인터 *pc를 만든다.
int *pi; //int형 포인터 *pi를 만든다.
float *pf; // float형 포인터 *pf를 만든다.
double *pd; //double형 포인터 *pd를 만든다.
char c; //문자형 변수 c를 만든다.
int i; //int 형 변수 i를 만든다.
float f; //float형 변수 f를 만든다.
double d; //double형 변수 d를 만든다.
pc = NULL; //pc값을 초기화한다. NULL값을 넣는다.
printf("pc = %lu\n", pc); //long unsigned 변수 형태를 출력하는 식이다.
*pc = 100; //*pc가 바라보는 곳에 100값을 넣는다. 바라보는 주소가 NULL이라 잘못된 식.
}
```

3.(3) 출력화면 및 설명

pc =NULL;을 해줘서 출력을 할 때 %lu와는 출력형태가 다르지만 0값이 출력이 되었다. long unsigned 형태일 때나 char 형태일때나 문자형 포인터일 때나 NULL값은 0으로 출력이 된다는 것을 알 수 있다. 따라서 출력하려는 것과 출력형태가 다르지만 0값이 출력되었다.

3.(4) 코드 및 주석

```
#include <stdio.h>
void main()
{
        char* pc; //문자열 포인터 *pc를 만든다.
        int* pi; //int형 포인터 *pi를 만든다.
        float* pf; // float형 포인터 *pf를 만든다.
        double* pd; //double형 포인터 *pd를 만든다.
        char c; //문자형 변수 c를 만든다.
        int i; //int 형 변수 i를 만든다.
        float f; //float형 변수 f를 만든다.
        double d; //double형 변수 d를 만든다.
        pc = NULL; //pc값을 초기화한다. NULL값을 넣는다.
        printf("초기화하기전의 pc = %lu\n", pc); //long unsigned 변수 형태를 출력하는 식이다.
        pc = &c; //문자형 변수 c의 주소를 pc에 넣는다.
        printf("초기화하기후의 pc = %lu\n", pc); //long unsigned 변수 형태를 출력하는 식이다.
        *pc = 100; //*pc가 바라보는 곳에 100값을 넣는다.
        printf("c = %d\n", c);
}
```

3.(4) 출력화면 및 설명

```
초기화하기전의 pc = 0
초기화하기후의 pc = 2276456228
c = 100

C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 14292개)이(가) 종료되었습니다(코드: 0개)
.
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]
를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

pc=NULL을 해줬기 때문에 첫 번째 줄에는 0값이 출력된다. c의 주소값을 pc에 넣어줘서 두 번째 줄이 c의 주소값이 출력된다. *pc에 100값을 넣어서 pc는 c의 주소를 가지고 있기 때문에 c에 100이 들어가게 되고, 결국 c=100이라는 값이 출력되게 된다.

3.(5) 코드 및 주석

```
#include <stdio.h>
void main()
{
	char* pc; //문자열 포인터 *pc를 만든다.
	int* pi; //int형 포인터 *pi를 만든다.
	float* pf; // float형 포인터 *pf를 만든다.
	double* pd; //double형 포인터 *pd를 만든다.
	char c; //문자형 변수 c를 만든다.
	int i; //int 형 변수 i를 만든다.
	float f; //float형 변수 f를 만든다.
	double d; //double형 변수 d를 만든다.

	pi = NULL; //pi값을 초기화한다. NULL값을 넣는다.
	printf("초기화하기전의 pi = %lu\n", pi); //long unsigned 변수 형태를 출력하는 식이다.
	pi = &c; //문자형 변수 c의 주소를 pi에 넣는다.
	printf("초기화하기후의 pi = %lu\n", pi); //long unsigned 변수 형태를 출력하는 식이다.
	*pi = 100; //*pi가 바라보는 곳에 100값을 넣는다.
	printf("c = %d\n", c);
	pf = NULL; //pf값을 초기화한다. NULL값을 넣는다.
	printf("초기화하기전의 pf = %lu\n", pf); //long unsigned 변수 형태를 출력하는 식이다.
	pf = &c; //문자형 변수 c의 주소를 pf에 넣는다.
	printf("초기화하기후의 pf = %lu\n", pf); //long unsigned 변수 형태를 출력하는 식이다.
	*pf = 100; //*pf가 바라보는 곳에 100값을 넣는다.
	printf("c = %d\n", c);
	pd = NULL; //pd값을 초기화한다. NULL값을 넣는다.
	printf("초기화하기전의 pd = %lu\n", pd); //long unsigned 변수 형태를 출력하는 식이다.
	pd = &c; //문자형 변수 c의 주소를 pd에 넣는다.
	printf("초기화하기후의 pd = %lu\n", pd); //long unsigned 변수 형태를 출력하는 식이다.
	*pd = 100; //*pd가 바라보는 곳에 100값을 넣는다.바라보는 주소가 NULL이라 잘못된 식.
	printf("c = %d\n", c);
}
```

3.(5) 출력화면 및 설명

```
초기화하기전의 pi = 0
초기화하기후의 pi = 3074160948
i = 100
초기화하기전의 pf = 0
초기화하기후의 pf = 3074160980
f = 100.000000
초기화하기전의 pd = 0
초기화하기후의 pd = 3074161016
d = 100.000000

C:\Users\lym69\OneDrive\바탕 화면\cording\x64\Debug\cording.exe(프로세스 13768개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

결과값이 3(4)와 차이가 없다. 첫 번째 줄은 초기화된 NULL값이 출력되었고, 초기화 하기 후는 각각 다르게 나온 것처럼 보이지만 모두 쓰레기값이 나온걸 확인할 수 있다. (c의 주소가 정해지지 않았기 때문이다.) I,f,d 자료형의 출력형태에 따라서 소수점이 있고 없고의 차이는 있지만, 모두 100의 값이 나온 것을 확인 할 수 있다.

3.(6) 코드 및 주석

```
#include <stdio.h>
void main(void)
{
int a[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
int *p;
p = a; /* p에 배열 a의 시작 주소를 치환한다. */
/* 이 문장은 포인터를 사용하여 배열 a의 첫째, 둘째, 셋째 원소들을 출력한다. */
printf("%d %d %d\n", *p, *(p+1), *(p+2));
/* 이 문장은 배열이름을 포인터처럼 사용하여 배열 a의 첫째, 둘째, 셋째 원소들을 출력한다. */
printf("%d %d %d\n", *a, *(a+1), *(a+2));
/* 이 문장은 배열 a를 사용하여 같은 것을 출력한다. */
printf("%d %d %d\n", a[0], a[1], a[2]);
}
```

3.(6) 출력화면 및 설명

```
10 20 30
10 20 30
10 20 30

C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 2952개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

첫 번째줄의 출력은 포인터 p가 배열 a의 주소를 바라보고 있기 때문에 *p는 a[0], *(p+1)은 a[1], *(p+2)은 a[2]를 출력한다. 두 번째줄의 출력은 *a가 배열 a의 첫 번째 주소와 같기 때문에 역시 같은 결과가 나온다. 따라서 3개의 줄이 모두 같은 결과값이 나온다.

3.(7) 코드 및 주석

```
#include <stdio.h>
void main(void)
{
char str[] = "What is Pointer?"; //배열 str에 문자열을 저장한다.
char *p;
int i;
p = str; //포인터 p가 배열str의 주소를 바라보게 된다.
/* 널 문자가 발견될 때까지 반복한다. */
for(i=0; p[i] != NULL ; i++) //배열의 끝까지 반복한다.
printf("%c",p[i]); //문자의 형태로 배열 str을 출력한다.
}
```

3.(7) 출력화면 및 설명

```
What is Pointer?
C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 15440개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

문자열을 받은 배열 str에서 p=str을 해서 p에 str의 주소값을 넣어서 포인터 *p가 배열 str의 주소를 바라보게 한다. 그 후 for문으로 p[i]의 형태로 문자를 하나씩 출력하고, 그 결과 “What is Pointer? “이 출력된다.

3.(8) 코드 및 주석

```
#include <stdio.h>
char *pa[] =        //문자열 형식의 포인터배열  *pa를 선언한다.
{
"에러 1",
"에러 2",
"에러 3",
"에러 4",
"에러 5",
"에러 6"
};
void error(int err_num) // 매개변수를 가지는 함수 error를 만든다.
{
printf(pa[err_num]); //main에서 error(1)을 받아왔기 때문에 pa[1]을 출력한다.
}
main()
{
error(1); //함수 error에 1값을 보낸다.
}
```

3.(8) 출력화면 및 설명

```
에러 2
C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 1604개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

char *pa[](문자열을 가지기 때문에 포인터배열이 가능하다.)를 선언하고, main함수에서 error(1)을 실행하면, error 함수에서 1값을 받아서, pa[1]을 출력한다. pa[1]은 “에러 2”이기 때문에 “에러 2”값이 출력된다.

3.(9) 코드 및 주석

```
#include <stdio.h >
void swap1(int i, int j); //i와 j를 매개변수로 가지는 함수 swap1을 선언한다.
void swap2(int *i, int *j);  //*i와 *j를 매개변수로 가지는 함수 swap2을 선언한다.
//
void main()
{
int num1, num2;
num1 = 100;
num2 = 200;
swap1(num1, num2);
printf("num1 : %d num2 : %d\n", num1, num2); //swap1을 실행한 후의 결과물이 출력된다.
swap2(&num1, &num2);
printf("num1 : %d num2 : %d\n", num1, num2); //swap2를 실행한 후의 결과물이 출력된다.
}
//
void swap1(int i, int j) //num1과 num2를 받아 i와 j에 복사한다.
{
int temp;
temp = i;
i = j;
j = temp; // temp라는 변수를 만든 후 변수 temp를 이용하여 i와 j의 값을 바꾼다.
}
//
void swap2(int *i, int *j) //num1의 주소와 num2의 주소를 포인터 i와 j에 복사한다.
{
int temp;
temp = *i;
*i = *j;
*j = temp;  //temp라는 변수를 만든 후 변수 temp를 이용하여 *i와 *j의 값을 바꾼다.
}
```

3.(9) 출력화면 및 설명

```
num1 : 100 num2 : 200
num1 : 200 num2 : 100

C:\Users\lym69\OneDrive\바탕 화면\Project1\x64\Debug\Project1.exe(프로세스 6652개)이(가) 종료되었습니다(코드: 0개).
디버깅이 중지될 때 콘솔을 자동으로 닫으려면 [도구] -> [옵션] -> [디버깅] > [디버깅이 중지되면 자동으로 콘솔 닫기]를 사용하도록 설정합니다.
이 창을 닫으려면 아무 키나 누르세요...
```

함수 swap1과 swap2를 만든 후, 하나는 그냥 매개변수를 사용하고, 하나는 포인터를 이용한 매개변수를 사용했는데, 그에 대한 결과의 차이를 출력하는 코드이다. 첫 번째 줄 출력은 일반 매개변수를 사용한 swap1이 적용된 코드인데, main함수의 변수 num1과 num2의 값이 변하지 않은 것을 알 수 있는데, 그러한 이유는 일반 매개변수를 전해주면 swap1 안에서 i과 j가 지역변수로 선언되어서 함수 내에서 변화가 있어도 main함수의 영향을 주지 않기 때문이다. 그에 반해, 두 번째 줄 출력은 결과 값이 변화한 것을 알 수 있다. 이러한 이유는 swap2가 포인터 매개변수를 사용한 함수이기 때문이다. 포인터를 사용하는 경우 swap2 내에서 변수의 값이 변하면 main함수에도 영향을 준다. 따라서 swap2함수에서 *i와 *j를 바꾸는 코드가 main함수에 적용이 되어서 결과값이 num1:200 num2: 100이라는 결과값이 나온다.